```python
#Lab 1
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

nltk.download("punkt_tab", quiet=True)
nltk.download("stopwords", quiet=True)

text = "Hello World! Year 2026"

print("Original Text:")
print(text)
print("-" * 50)

tokens = word_tokenize(text)
print("Tokens:")
print(tokens)
print("-" * 50)

stop_words = set(stopwords.words('english'))
filtered_tokens = []

for word in tokens:

    word = word.lower()

    if word.isalpha() and word not in stop_words:
        filtered_tokens.append(word)

print("Filtered tokens:")
print(filtered_tokens)

print("-" * 50)

def is_english_script(text):

    pattern = re.compile(r'^[A-Za-z0-9\s.,!?\'-]+$')
    return bool(pattern.match(text))

if is_english_script(text):
    print("Script Validation: Text is English (Latin script)")
else:
    print("Script Validation: Text contains invalid script")
```

```
Original Text:
Hello World! Year 2026
--------------------------------------------------
Tokens:
['Hello', 'World', '!', 'Year', '2026']
--------------------------------------------------
```

```
Filtered tokens:
['hello', 'world', 'year']
--------------------------------------------------
Script Validation: Text is English (Latin script)
```

```python
#Lab 2
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords, wordnet
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag

# Download required resources
nltk.download("punkt")
nltk.download("stopwords")
nltk.download("averaged_perceptron_tagger")
nltk.download("wordnet")

text = "Android is better than Apple"

print("=" * 60)
print("CORRECT NLP PREPROCESSING PIPELINE")
print("=" * 60)

# Step 1: Original Input
print("\n1. Original Input Text:")
print(text)

# Step 2: Tokenization
tokens = word_tokenize(text)

print("\n2. Tokenized Words:")
print(tokens)

# Step 3: POS Tagging (Before Stopword Removal)
tagged_tokens = pos_tag(tokens)

print("\n3. POS Tagging:")
print(tagged_tokens)

# Step 4: Stopword Removal
stop_words = set(stopwords.words("english"))

filtered_tokens = [
    (word, tag) for word, tag in tagged_tokens
    if word.lower() not in stop_words and word.isalpha()
]

print("\n4. After Stopword Removal:")
print(filtered_tokens)
```

```python
# Step 5: Stemming
stemmer = PorterStemmer()

stemmed_words = [
    stemmer.stem(word) for word, tag in filtered_tokens
]

print("\n5. Stemmed Words:")
print(stemmed_words)

# Step 6: Convert POS tags for WordNet
def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith("J"):
        return wordnet.ADJ
    elif treebank_tag.startswith("V"):
        return wordnet.VERB
    elif treebank_tag.startswith("N"):
        return wordnet.NOUN
    elif treebank_tag.startswith("R"):
        return wordnet.ADV
    else:
        return wordnet.NOUN

# Step 7: Lemmatization
lemmatizer = WordNetLemmatizer()

lemmatized_words = [
    lemmatizer.lemmatize(word, get_wordnet_pos(tag))
    for word, tag in filtered_tokens
]

print("\n6. Lemmatized Words:")
print(lemmatized_words)

print("\n" + "=" * 60)
print("PIPELINE COMPLETED")
print("=" * 60)
```

```
============================================================
CORRECT NLP PREPROCESSING PIPELINE
============================================================

1. Original Input Text:
Android is better than Apple

2. Tokenized Words:
['Android', 'is', 'better', 'than', 'Apple']

3. POS Tagging:
[('Android', 'NNP'), ('is', 'VBZ'), ('better', 'JJR'), ('than', 'IN'),
```

```
('Apple', 'NNP')]

4. After Stopword Removal:
[('Android', 'NNP'), ('better', 'JJR'), ('Apple', 'NNP')]

5. Stemmed Words:
['android', 'better', 'appl']

6. Lemmatized Words:
['Android', 'good', 'Apple']

============================================================
PIPELINE COMPLETED
============================================================

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Mohit\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Mohit\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\Mohit\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Mohit\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```python
#Lab 3
from sklearn.feature_extraction.text import TfidfVectorizer,
CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import pairwise_distances
import pandas as pd

# Documents
d0 = "This is a sample notebook"
d1 = "This notebook is sample"
d2 = "This is another sample notebook"

docs = [d0, d1, d2]

# ---------------- TF-IDF ----------------
print("=" * 60)
print("TF-IDF ANALYSIS")
print("=" * 60)

tfidf = TfidfVectorizer()
result = tfidf.fit_transform(docs)
```

```python
print("\nIDF Values:")
for word, val in zip(tfidf.get_feature_names_out(), tfidf.idf_):
    print(f"{word:10} : {val:.4f}")

print("\nWord Indexes:")
print(tfidf.vocabulary_)

print("\nTF-IDF Matrix:")
print(result.toarray())

# ---------------- COUNT VECTOR ----------------
print("\n" + "=" * 60)
print("DOCUMENT TERM MATRIX")
print("=" * 60)

cv = CountVectorizer()
matrix = cv.fit_transform(docs)

dtm_df = pd.DataFrame(
    matrix.toarray(),
    columns=cv.get_feature_names_out(),
    index=["d0", "d1", "d2"]
)

print(dtm_df)

# ---------------- COSINE ----------------
print("\n" + "=" * 60)
print("COSINE SIMILARITY")
print("=" * 60)

cosine_sim = cosine_similarity(matrix)

cosine_df = pd.DataFrame(
    cosine_sim,
    index=["d0", "d1", "d2"],
    columns=["d0", "d1", "d2"]
)

print(cosine_df)

# ---------------- JACCARD ----------------
print("\n" + "=" * 60)
print("JACCARD SIMILARITY")
print("=" * 60)

binary_cv = CountVectorizer(binary=True, token_pattern=r"(?u)\b\w+\b")
X = binary_cv.fit_transform(docs)

jaccard_dist = pairwise_distances(X.toarray(), metric="jaccard")
```

```python
jaccard_sim = 1 - jaccard_dist

jaccard_df = pd.DataFrame(
    jaccard_sim,
    index=["d0", "d1", "d2"],
    columns=["d0", "d1", "d2"]
)

print(jaccard_df)
```

```
================================================================
TF-IDF ANALYSIS
================================================================

IDF Values:
another    : 1.6931
is         : 1.0000
notebook   : 1.0000
sample     : 1.0000
this       : 1.0000

Word Indexes:
{'this': 4, 'is': 1, 'sample': 3, 'notebook': 2, 'another': 0}

TF-IDF Matrix:
[[0.         0.5        0.5        0.5        0.5       ]
 [0.         0.5        0.5        0.5        0.5       ]
 [0.64612892 0.38161415 0.38161415 0.38161415 0.38161415]]

================================================================
DOCUMENT TERM MATRIX
================================================================
    another  is  notebook  sample  this
d0        0   1         1       1     1
d1        0   1         1       1     1
d2        1   1         1       1     1

================================================================
COSINE SIMILARITY
================================================================
          d0        d1        d2
d0  1.000000  1.000000  0.894427
d1  1.000000  1.000000  0.894427
d2  0.894427  0.894427  1.000000

================================================================
JACCARD SIMILARITY
================================================================
          d0   d1        d2
d0  1.000000  0.8  0.666667
```

```
d1  0.800000  1.0  0.800000
d2  0.666667  0.8  1.000000
```

```
E:\Anaconda3\Lib\site-packages\sklearn\metrics\pairwise.py:2462:
DataConversionWarning: Data was converted to boolean for metric
jaccard
  warnings.warn(msg, DataConversionWarning)
```

```
#Lab 4(Installation)
!pip install vaderSentiment
```

```
Requirement already satisfied: vaderSentiment in e:\anaconda3\lib\
site-packages (3.3.2)
Requirement already satisfied: requests in e:\anaconda3\lib\site-
packages (from vaderSentiment) (2.32.5)
Requirement already satisfied: charset_normalizer<4,>=2 in e:\
anaconda3\lib\site-packages (from requests->vaderSentiment) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in e:\anaconda3\lib\site-
packages (from requests->vaderSentiment) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in e:\anaconda3\lib\
site-packages (from requests->vaderSentiment) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in e:\anaconda3\lib\
site-packages (from requests->vaderSentiment) (2026.4.22)
```

```
# lab4_sentimentanalysis_fianance
import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.metrics import classification_report, accuracy_score

def analyze_finance_data():
    analyzer = SentimentIntensityAnalyzer()

    print("=" * 70)
    print("Loading Finance Dataset...")
    print("=" * 70)

    try:
        df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
data/lab5_sentiment_finance.csv", encoding="latin-1").dropna()
    except FileNotFoundError:
        print("Error: lab5_sentiment_finance.csv not found.")
        return

    # Prediction Function
    def get_vader_label(text):
        score = analyzer.polarity_scores(str(text).lower())
["compound"]

        if score >= 0.05:
            return "Positive"
```

```
        elif score <= -0.05:
            return "Negative"
        else:
            return "Neutral"

    print("Applying VADER Sentiment Analysis...")
    df["vader_prediction"] = df["text"].apply(get_vader_label)

    # Format true labels
    df["true_label"] = df["category"].str.capitalize()

    valid_df = df.dropna(subset=["true_label", "vader_prediction"])

    # Sample Predictions
    print("\n" + "=" * 70)
    print("Sample Predictions")
    print("=" * 70)

    for i, row in valid_df.head(5).iterrows():
        print(f"\nNews {i+1}:")
        print(f"Text       : {str(row['text'])[:120]}...")
        print(f"Predicted  : {row['vader_prediction']}")

    # Accuracy
    acc = accuracy_score(valid_df["true_label"],
valid_df["vader_prediction"])

    print("\n" + "=" * 70)
    print("Model Performance")
    print("=" * 70)
    print(f"Accuracy : {acc:.4f}")

    print("\nClassification Report:")
    print(classification_report(valid_df["true_label"],
valid_df["vader_prediction"]))

if __name__ == "__main__":
    analyze_finance_data()
```

```
======================================================================
Loading Finance Dataset...
======================================================================
Applying VADER Sentiment Analysis...

======================================================================
Sample Predictions
======================================================================

News 1:
Text       : According to Gran , the company has no plans to move all
production to Russia , although that is where the company is gr...
```

```
Predicted  : Negative

News 2:
Text       : Technopolis plans to develop in stages an area of no less
than 100,000 square meters in order to host companies working ...
Predicted  : Negative

News 3:
Text       : The international electronic industry company Elcoteq has
laid off tens of employees from its Tallinn facility ; contrar...
Predicted  : Neutral

News 4:
Text       : With the new production plant the company would increase
its capacity to meet the expected increase in demand and would ...
Predicted  : Positive

News 5:
Text       : According to the company 's updated strategy for the
years 2009-2012 , Basware targets a long-term net sales growth in t...
Predicted  : Positive

======================================================================
Model Performance
======================================================================
Accuracy : 0.5433

Classification Report:
              precision    recall  f1-score   support

    Negative       0.41      0.30      0.34       604
     Neutral       0.74      0.52      0.61      2879
    Positive       0.40      0.71      0.51      1363

    accuracy                           0.54      4846
   macro avg       0.52      0.51      0.49      4846
weighted avg       0.60      0.54      0.55      4846
```

```python
# lab4_sentimentanalysis_sentimentdata
import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.metrics import classification_report, accuracy_score

def analyze_general_sentiment_data():
    analyzer = SentimentIntensityAnalyzer()

    print("=" * 70)
    print("Loading General Sentiment Dataset...")
    print("=" * 70)
```

```
    try:
        df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
data/lab5_sentiment_data.csv", encoding="latin-1").dropna()
    except FileNotFoundError:
        print("Error: lab5_sentiment_data.csv not found.")
        return

    # Label Mapping
    sentiment_map = {
        0: "Negative",
        1: "Neutral",
        2: "Positive"
    }

    df["true_label"] = df["Sentiment"].map(sentiment_map)

    # Prediction Function
    def get_vader_label(text):
        score = analyzer.polarity_scores(str(text).lower())
["compound"]

        if score >= 0.05:
            return "Positive"
        elif score <= -0.05:
            return "Negative"
        else:
            return "Neutral"

    print("Applying VADER Sentiment Analysis...")
    df["vader_prediction"] = df["text"].apply(get_vader_label)

    valid_df = df.dropna(subset=["true_label", "vader_prediction"])

    # Sample Output
    print("\n" + "=" * 70)
    print("Sample Predictions")
    print("=" * 70)

    for i, row in valid_df.head(5).iterrows():
        print(f"\nText {i+1}:")
        print(f"Message    : {str(row['text'])[:120]}...")
        print(f"Predicted  : {row['vader_prediction']}")

    # Accuracy
    acc = accuracy_score(valid_df["true_label"],
valid_df["vader_prediction"])

    print("\n" + "=" * 70)
    print("Model Performance")
```

```
    print("=" * 70)
    print(f"Accuracy : {acc:.4f}")

    print("\nClassification Report:")
    print(classification_report(valid_df["true_label"],
valid_df["vader_prediction"]))

if __name__ == "__main__":
    analyze_general_sentiment_data()
```

```
======================================================================
Loading General Sentiment Dataset...
======================================================================
Applying VADER Sentiment Analysis...

======================================================================
Sample Predictions
======================================================================

Text 1:
Message    : lets forget apple pay required brand new iphone order use
significant portion apples user base wasnt able use even wante...
Predicted  : Positive

Text 2:
Message    : nz retailers don?t even contactless credit card machines
like paywave support apple pay don?t like high fees come...
Predicted  : Positive

Text 3:
Message    : forever acknowledge channel help lessons ideas
explanations quite helpful youll sit comfort monitor account growth...
Predicted  : Positive

Text 4:
Message    : whenever go place doesn?t take apple pay doesn?t happen
often it?s drag ?contactless covid? habits getting apple card i?...
Predicted  : Negative

Text 5:
Message    : apple pay convenient secure easy use used korean japanese
airports need physical credit cards...
Predicted  : Positive

======================================================================
Model Performance
======================================================================
Accuracy : 0.7036

Classification Report:
```

```
              precision    recall  f1-score   support

    Negative       0.52      0.52      0.52      1364
     Neutral       0.57      0.42      0.48      2468
    Positive       0.78      0.86      0.82      6163

    accuracy                           0.70      9995
   macro avg       0.62      0.60      0.60      9995
weighted avg       0.69      0.70      0.69      9995
```

```python
# lab5_tweetsentiment
import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.metrics import classification_report, accuracy_score

def analyze_twitter_data():
    analyzer = SentimentIntensityAnalyzer()

    print("=" * 70)
    print("Loading Twitter Dataset...")
    print("=" * 70)

    try:
        df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
data/lab5_Twitter_Data_sentiment.csv", encoding="latin-1").dropna()
    except FileNotFoundError:
        print("Error: Dataset file not found.")
        return

    # Label Mapping
    category_map = {
        -1.0: "Negative",
         0.0: "Neutral",
         1.0: "Positive"
    }

    df["true_label"] = df["category"].map(category_map)

    # VADER Prediction Function
    def get_vader_label(text):
        score = analyzer.polarity_scores(str(text).lower())
["compound"]

        if score >= 0.05:
            return "Positive"
        elif score <= -0.05:
            return "Negative"
        else:
```

```
            return "Neutral"

    print("Applying VADER Sentiment Analysis...")
    df["vader_prediction"] = df["text"].apply(get_vader_label)

    valid_df = df.dropna(subset=["true_label", "vader_prediction"])

    # Sample Results
    print("\n" + "=" * 70)
    print("Sample Predictions")
    print("=" * 70)

    for i, row in valid_df.head(5).iterrows():
        print(f"\nTweet {i+1}:")
        print(f"Text       : {str(row['text'])[:120]}...")
        print(f"Predicted  : {row['vader_prediction']}")

    # Accuracy
    acc = accuracy_score(valid_df["true_label"],
valid_df["vader_prediction"])

    print("\n" + "=" * 70)
    print("Model Performance")
    print("=" * 70)
    print(f"Accuracy : {acc:.4f}")

    # Classification Report
    print("\nClassification Report:")
    print(classification_report(valid_df["true_label"],
valid_df["vader_prediction"]))

if __name__ == "__main__":
    analyze_twitter_data()
```

```
======================================================================
Loading Twitter Dataset...
======================================================================
Applying VADER Sentiment Analysis...

======================================================================
Sample Predictions
======================================================================

Tweet 1:
Text       : when modi promised ?minimum government maximum
governance? expected him begin the difficult job reforming the state
why ...
Predicted  : Positive

Tweet 2:
Text       : talk all the nonsense and continue all the drama will
```

```
vote for modi ...
Predicted  : Negative

Tweet 3:
Text       : what did just say vote for modi  welcome bjp told you
rahul the main campaigner for modi think modi should just relax...
Predicted  : Positive

Tweet 4:
Text       : asking his supporters prefix chowkidar their names modi
did great service now there confusion what read what not now cru...
Predicted  : Negative

Tweet 5:
Text       : answer who among these the most powerful world leader
today trump putin modi may ...
Predicted  : Positive

======================================================================
Model Performance
======================================================================
Accuracy : 0.5647

Classification Report:
              precision    recall  f1-score   support

    Negative       0.45      0.60      0.52      2370
     Neutral       0.66      0.41      0.51      3476
    Positive       0.60      0.67      0.63      4153

    accuracy                           0.56      9999
   macro avg       0.57      0.56      0.55      9999
weighted avg       0.58      0.56      0.56      9999
```

```python
# lab7_pagerank_git
import pandas as pd
import networkx as nx

def run_lab7_github():
    print("Loading GitHub Dataset...")
    df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
data/lab8_github_edges.csv")

    print("Building Directed Graph...")
    G = nx.from_pandas_edgelist(
        df,
        'source',
        'target',
```

```
        create_using=nx.DiGraph()
    )

    print(f"Nodes: {G.number_of_nodes()} | Edges:
{G.number_of_edges()}")

    print("Computing PageRank (alpha=0.85)...")
    pagerank = nx.pagerank(G, alpha=0.85)

    sorted_pr = sorted(pagerank.items(), key=lambda x: x[1],
reverse=True)

    print("\n--- Top 10 GitHub Nodes by PageRank ---")
    for i, (page, rank) in enumerate(sorted_pr[:10]):
        print(f"Rank {i+1}: Node {page} = {rank:.6f}")

if __name__ == "__main__":
    run_lab7_github()
```

```
Loading GitHub Dataset...
Building Directed Graph...
Nodes: 37700 | Edges: 289003
Computing PageRank (alpha=0.85)...

--- Top 10 GitHub Nodes by PageRank ---
Rank 1: Node 31890 = 0.019439
Rank 2: Node 36652 = 0.013467
Rank 3: Node 18163 = 0.010761
Rank 4: Node 36628 = 0.009649
Rank 5: Node 35008 = 0.005277
Rank 6: Node 34114 = 0.004990
Rank 7: Node 2347 = 0.004469
Rank 8: Node 37471 = 0.004384
Rank 9: Node 35773 = 0.003695
Rank 10: Node 37289 = 0.003422
```

```
# lab7_pagerank_higgs
import pandas as pd
import networkx as nx

def run_lab7_higgs():
    print("Loading Higgs Twitter Dataset...")
    df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
data/lab8_higgs-social_network.csv")

    print("Building Directed Graph...")
    G = nx.from_pandas_edgelist(
        df,
        'source',
```

```
        'target',
        create_using=nx.DiGraph()
    )

    print(f"Nodes: {G.number_of_nodes()} | Edges:
{G.number_of_edges()}")

    print("Computing PageRank (alpha=0.85)...")
    pagerank = nx.pagerank(G, alpha=0.85)

    sorted_pr = sorted(pagerank.items(), key=lambda x: x[1],
reverse=True)

    print("\n--- Top 10 Higgs Nodes by PageRank ---")
    for i, (page, rank) in enumerate(sorted_pr[:10]):
        print(f"Rank {i+1}: Node {page} = {rank:.6f}")

if __name__ == "__main__":
    run_lab7_higgs()
```

```
Loading Higgs Twitter Dataset...
Building Directed Graph...
Nodes: 5535 | Edges: 10001
Computing PageRank (alpha=0.85)...

--- Top 10 Higgs Nodes by PageRank ---
Rank 1: Node 220 = 0.000340
Rank 2: Node 32 = 0.000334
Rank 3: Node 6 = 0.000315
Rank 4: Node 38 = 0.000302
Rank 5: Node 8 = 0.000288
Rank 6: Node 77 = 0.000286
Rank 7: Node 298 = 0.000284
Rank 8: Node 50 = 0.000282
Rank 9: Node 2 = 0.000282
Rank 10: Node 46 = 0.000279
```

```
# Lab7_pagerank_fb
import pandas as pd
import networkx as nx

def run_lab7_facebook():
    print("Loading Facebook Dataset...")
    df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
data/lab8_github_edges.csv")

    # Create Undirected Graph (Facebook Friendship Network)
    print("Building Undirected Graph...")
    G = nx.from_pandas_edgelist(
```

```
        df,
        'source',
        'target',
        create_using=nx.Graph()
    )

    print(f"Nodes: {G.number_of_nodes()} | Edges:
{G.number_of_edges()}")

    # Apply PageRank
    print("Computing PageRank (alpha=0.85)...")
    pagerank = nx.pagerank(G, alpha=0.85)

    # Sort ranking
    sorted_pr = sorted(pagerank.items(), key=lambda x: x[1],
reverse=True)

    # Display Top 10
    print("\n--- Top 10 Facebook Nodes by PageRank ---")
    for i, (page, rank) in enumerate(sorted_pr[:10]):
        print(f"Rank {i+1}: Node {page} = {rank:.6f}")

if __name__ == "__main__":
    run_lab7_facebook()
```

```
Loading Facebook Dataset...
Building Undirected Graph...
Nodes: 37700 | Edges: 289003
Computing PageRank (alpha=0.85)...

--- Top 10 Facebook Nodes by PageRank ---
Rank 1: Node 31890 = 0.016514
Rank 2: Node 27803 = 0.011451
Rank 3: Node 35773 = 0.004919
Rank 4: Node 19222 = 0.004669
Rank 5: Node 13638 = 0.003875
Rank 6: Node 36652 = 0.003799
Rank 7: Node 18163 = 0.003381
Rank 8: Node 9051 = 0.002722
Rank 9: Node 10001 = 0.002665
Rank 10: Node 35008 = 0.002338
```

```
# Lab8_Centrality_Fb
import pandas as pd
import networkx as nx

def run_lab8_facebook():
    print("Loading Facebook Dataset...")
    df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
```

```
data/lab8_facebook_edges.csv")

    # Create Undirected Graph (best for Facebook friendships)
    print("Building Undirected Graph...")
    G = nx.from_pandas_edgelist(
        df,
        source="source",
        target="target",
        create_using=nx.Graph()
    )

    # Basic Info
    print("\nGraph Summary:")
    print("Total Nodes:", G.number_of_nodes())
    print("Total Edges:", G.number_of_edges())

    # Compute Degree Centrality
    print("\nComputing Degree Centrality...")
    degree = nx.degree_centrality(G)

    # Compute Betweenness Centrality
    print("Computing Betweenness Centrality (Approximation k=100)...")
    betweenness = nx.betweenness_centrality(G, k=100, seed=42)

    # Sort Top 5
    top_degree = sorted(degree.items(), key=lambda x: x[1],
reverse=True)[:5]
    top_between = sorted(betweenness.items(), key=lambda x: x[1],
reverse=True)[:5]

    # Display Results
    print("\n--- Top 5 Most Connected Users (Degree Centrality) ---")
    for node, val in top_degree:
        print(f"Node {node}: {val:.5f}")

    print("\n--- Top 5 Best Bridge Users (Betweenness Centrality)
---")
    for node, val in top_between:
        print(f"Node {node}: {val:.5f}")

if __name__ == "__main__":
    run_lab8_facebook()
```

```
Loading Facebook Dataset...
Building Undirected Graph...

Graph Summary:
Total Nodes: 4039
Total Edges: 88234
```

```
Computing Degree Centrality...
Computing Betweenness Centrality (Approximation k=100)...

--- Top 5 Most Connected Users (Degree Centrality) ---
Node 107: 0.25879
Node 1684: 0.19614
Node 1912: 0.18697
Node 3437: 0.13546
Node 0: 0.08593

--- Top 5 Best Bridge Users (Betweenness Centrality) ---
Node 107: 0.49713
Node 1684: 0.34584
Node 1912: 0.23593
Node 3437: 0.18749
Node 0: 0.15731
```

```python
# lab8_centrality_git
import pandas as pd
import networkx as nx

def run_lab8_github():
    print("Loading GitHub Dataset...")
    df =
pd.read_csv("C:/Users/Mohit/Downloads/ASTMA_lab-main/ASTMA_lab-main/
data/lab8_github_edges.csv")

    # Create directed graph
    print("Building Directed Graph...")
    G = nx.from_pandas_edgelist(df, 'source', 'target',
create_using=nx.DiGraph())

    # Compute Metrics
    print("Computing Degree Centrality...")
    degree = nx.degree_centrality(G)

    print("Computing Betweenness Centrality (Approximation k=100)...")
    betweenness = nx.betweenness_centrality(G, k=100, seed=42)

    # Sort and Display Top 5
    top_degree = sorted(degree.items(), key=lambda x: x[1],
reverse=True)[:5]
    top_between = sorted(betweenness.items(), key=lambda x: x[1],
reverse=True)[:5]

    print("\n--- GitHub Top 5: Degree Centrality (Most Connected)
---")
    for node, val in top_degree: print(f"Node {node}: {val:.5f}")

    print("\n--- GitHub Top 5: Betweenness Centrality (Best
```

```
Connectors) ---")
    for node, val in top_between: print(f"Node {node}: {val:.5f}")

if __name__ == "__main__":
    run_lab8_github()
```

```
Loading GitHub Dataset...
Building Directed Graph...
Computing Degree Centrality...
Computing Betweenness Centrality (Approximation k=100)...

--- GitHub Top 5: Degree Centrality (Most Connected) ---
Node 31890: 0.25088
Node 27803: 0.18794
Node 35773: 0.08817
Node 19222: 0.07846
Node 13638: 0.06547

--- GitHub Top 5: Betweenness Centrality (Best Connectors) ---
Node 31890: 0.03508
Node 19222: 0.02185
Node 9051: 0.01311
Node 35773: 0.01281
Node 13638: 0.01193
```

```
# lab8_centrality_higgs
import pandas as pd
import networkx as nx

def run_lab8_higgs():
    print("Loading Higgs Dataset...")
    df = pd.read_csv("C:/Users/KUNAL SHETIYA/Astma-LCA3/lab8_higgs-
social_network.csv")

    # Create directed graph
    print("Building Directed Graph...")
    G = nx.from_pandas_edgelist(df, 'source', 'target',
create_using=nx.DiGraph())

    # Compute Metrics
    print("Computing Degree Centrality...")
    degree = nx.degree_centrality(G)

    print("Computing Betweenness Centrality (Approximation k=100)...")
    betweenness = nx.betweenness_centrality(G, k=100, seed=42)

    # Sort and Display Top 5
    top_degree = sorted(degree.items(), key=lambda x: x[1],
reverse=True)[:5]
    top_between = sorted(betweenness.items(), key=lambda x: x[1],
```

```
reverse=True)[:5]

    print("\n--- Higgs Top 5: Degree Centrality (Most Connected) ---")
    for node, val in top_degree: print(f"Node {node}: {val:.5f}")

    print("\n--- Higgs Top 5: Betweenness Centrality (Best Connectors)
---")
    for node, val in top_between: print(f"Node {node}: {val:.5f}")

if __name__ == "__main__":
    run_lab8_higgs()
```